

Diya Meditation Kiosk

Codebase Overview & Technical Documentation

An unattended meditation kiosk for a public / museum setting. A visitor is identified, a camera/CV + meditation pipeline runs, a PDF report is shown on screen, and the kiosk resets for the next person.

Application version 1.5.0

Kiosk app (C#/NET 8/Avalonia) + Backend (Node/Express/Postgres)

1. What This Project Is

Diya is an unattended meditation kiosk. A person is identified at the kiosk, a pipeline runs cameras / computer-vision scripts and a meditation session, a PDF report is generated, and that report is displayed on the kiosk. The machine then resets for the next visitor.

The system is split across two teams. This codebase covers OUR side: the desktop kiosk application, the registration / admin web backend, and packaging. The camera / servo / CV scripts and the separate 'meditation-app' that generates the report PDF are provided by the hardware team; our app calls them and displays the result.

The three parts

- Kiosk desktop app (folder: DiyaMeditation/) - C#/.NET 8 + Avalonia. Runs fullscreen on Ubuntu, packaged as a .deb. This is the on-screen experience.
- Backend + web (folders: server/ and registration/) - Node/Express API + Postgres, plus static HTML pages. Hosted on Render. Handles registration, the admin roster, and linking a visitor to a kiosk session.
- External meditation-app (separate .deb, another team) - runs the actual meditation, writes a report PDF to /opt/meditation-app/data. Our app only launches it and reads its output.

2. End-to-End Flow

The runtime flow, step by step:

- 1. The kiosk boots and opens fullscreen (single screen: HomeView).
- 2. The kiosk asks the backend for a session (POST /api/sessions) and shows that session token as a QR code on screen.
- 3. Beforehand, an admin has uploaded an Excel roster at /admin; every person in it received a personal login link of the form <site>/p/<token>.
- 4. A visitor opens their personal link on their phone, taps 'Proceed to login', which opens the phone camera as a scanner.
- 5. The visitor scans the QR shown on the kiosk. Their phone sends person + session to the backend (POST /api/claim), which links their identity to that kiosk session.
- 6. The kiosk has been polling (GET /api/sessions/:token). It sees the session is 'claimed', shows the visitor's name and downloads their photo.
- 7. On successful identification the kiosk AUTOMATICALLY runs the pipeline script (scripts/run1.sh).
- 8. run1.sh runs the camera/CV steps HOME1 -> SHOOT1 -> CHEST1 -> EYE1 (with retry logic), then launches the external meditation-app.
- 9. meditation-app runs the session and writes a report PDF into /opt/meditation-app/data.
- 10. When run1.sh finishes, the kiosk finds the newest PDF in that folder and renders it inside the app.
- 11. A 'Return' button resets everything back to step 2 for the next person.

3. Flow Chart



Note: steps 3-5 happen on the visitor's phone / an admin PC; steps 1-2 and 6-11 happen on the kiosk itself.

4. Technology Stack

- Kiosk app: C# / .NET 8 (LTS), Avalonia 11.2.3 (cross-platform UI that supports Linux).
- QR generation on the kiosk: QRCoder.
- In-app PDF rendering: PDFtoImage 4.1.1 (PDFium under the hood; renders PDF pages to images).
- Backend: Node.js + Express + Postgres (pg driver). Hosted on Render.
- Web pages: plain HTML/JJS. Admin page parses Excel in-browser with SheetJS; the phone login page uses html5-qrcode for the camera scanner.
- Packaging: self-contained single-file Debian .deb (bundles the .NET runtime + native libs).
- Pipeline scripts: bash (run1.sh) + Python 3.10 (the camera/CV scripts, provided by the hardware team).

5. Repository Structure

```

DiyaMeditation/      <- the kiosk desktop app (C#/Avalonia)
  Program.cs          app entry point
  App.axaml(.cs)      application + theme setup
  DiyaMeditation.csproj project / dependencies / version
  app.manifest        Windows-only manifest (unused on Linux)
  Models/            VisitorData, VisitorApiClient, VisitorQr
  Services/          PipelineRunner, ReportRenderer
  Views/             MainWindow (shell), HomeView (main screen)
  scripts/run1.sh     the editable pipeline script
  deploy/            build-deb.sh, setup-kiosk.sh, service, .desktop
  docker/            browser-preview image (dev only)
server/              the backend API (Node/Express)
  server.js db.js schema.sql package.json .env.example
registration/        static web pages
  index.html admin.html scan.html
render.yaml          Render deployment blueprint
PROJECT.md SETUP.md FAQ.md project documentation

```

6. File-by-File Reference

6.1 Kiosk app (DiyaMeditation/)

Program.cs

The .NET entry point. Configures Avalonia (platform auto-detect, Inter font, trace logging) and starts the classic desktop application lifetime, which loads App.

App.axaml

Application-level XAML. Loads the Fluent theme in its Light variant.

App.axaml.cs

On startup sets the single top-level window (MainWindow) and sets ShutdownMode = OnExplicitShutdown, so the process stays alive even if the window is closed.

DiyaMeditation.csproj

Project file. Targets .NET 8. References Avalonia 11.2.3, QRCoder (QR codes) and PDFtoImage 4.1.1 (report PDF rendering). InvariantGlobalization drops the libicu dependency so the .deb installs on any Ubuntu. IncludeNativeLibrariesForSelfExtract bundles the PDFium and SkiaSharp native libraries inside the single-file build. Version is 1.5.0. Bundles scripts/*.sh next to the app.

app.manifest

A Windows-only application manifest. Has no effect on the Linux kiosk target; kept because the template expects it.

Views/MainWindow.axaml

The kiosk shell window. Opens fullscreen. It is shown in the taskbar and is NOT top-most, so the app can be minimised and other windows are reachable. Hosts the current screen (HomeView) inside a content area.

Views/MainWindow.axaml.cs

Applies fullscreen at startup and re-asserts it a few times (a Wayland/GNOME quirk). The window can now be closed (Alt+F4) and minimised normally; the earlier close-veto and the secret exit shortcut have been removed.

Views/HomeView.axaml

The main screen layout. Left panel: 'Welcome', the visitor's details (circular photo, name, email, age), a Start button and a status line. Right panel: the on-screen QR, a live status line, a manual name/email/age fallback, and a 'New code' button. Also contains a full-screen report overlay (a scrollable list of PDF page images plus a 'Return' button).

Views/HomeView.axaml.cs

The core kiosk logic. Creates a backend session and renders its QR; polls every 2 seconds for a claim; when claimed, fills in the visitor details and downloads their photo; then AUTOMATICALLY runs the pipeline; when the pipeline finishes it locates and renders the newest report PDF into the overlay; the 'Return' button resets for the next visitor. Also handles the manual-entry fallback path.

Models/VisitorData.cs

A simple data object for a visitor: Name, Email, Age, and ImageUrl (mapped from the API's image_url field).

Models/VisitorApiClient.cs

The HTTP client for the backend. Base URL comes from the DIYA_API_BASE environment variable. Provides CreateSessionAsync (start a session), GetSessionAsync (poll for a claim), FetchAsync (legacy lookup by id), and DownloadBytesAsync (fetch the visitor photo). Uses a 60-second timeout to tolerate the hosting provider's cold starts.

Models/VisitorQr.cs

Legacy QR parsing from an earlier offline phase (decoded an embedded DIYA1:<base64 JSON> payload, or extracted a short id). Not used by the current flow; kept for reference.

Services/PipelineRunner.cs

Runs the bundled scripts/run1.sh using bash and WAITS for it to finish (there is intentionally no timeout - the script owns its own retry logic). Streams the script output for logging. The script path and shell are overridable via `DIYA_PIPELINE_SCRIPT` and `DIYA_BASH`.

Services/ReportRenderer.cs

Finds the newest *.pdf in the report directory (default `/opt/meditation-app/data`, overridable with `DIYA_REPORT_DIR`) and renders each page to an image (via `PDFtoImage/PDFium`) so the report can be shown inside the app - no external PDF viewer.

scripts/run1.sh

The editable pipeline. Runs the camera/CV Python scripts `HOME1/SHOOT1/CHEST1/EYE1` (from `~/Desktop/mark1`, using `python3.10`) with retry logic, then launches the external meditation-app which produces the report PDF. It first sanity-checks that the scripts and meditation-app exist. Editing this file (and rebuilding the .deb) changes the pipeline behaviour.

deploy/build-deb.sh

Builds a self-contained single-file .deb for amd64 or arm64 (default version 1.5.0). Bundles the .NET runtime and native libraries. Uses `dpkg-deb`, or an `ar/tar` fallback if it is absent.

deploy/setup-kiosk.sh

Installs the published app to `/opt`, installs and enables the systemd user service, enables 'lingering' so it runs without a login, prepares `/opt/meditation-app/data` (owned by the kiosk user so meditation-app can write and the kiosk can read), and disables GNOME shortcuts that could let a visitor escape.

deploy/diya-meditation.service

A systemd USER service. Starts with the graphical session, restarts automatically if the app exits (`Restart=always`), and carries the `DIYA_API_BASE` environment variable.

deploy/diya-meditation.desktop

A desktop autostart entry (used as an alternative to `/` alongside the systemd service).

docker/Dockerfile + entrypoint.sh

A browser-preview image only: runs the app on a virtual display and exposes it via `noVNC` on port 8080. For previewing the UI in a browser - NOT the real kiosk deployment.

.gitignore / .dockerignore

Standard ignore lists (build output, etc.).

6.2 Backend server (server/)**server.js**

The Express API and static-site server. Endpoints (see section 8) for sessions, visitors, the admin roster upload, per-person lookup, and claiming a session. The admin upload is protected by an `ADMIN_KEY`; header matching for the sheet columns is done loosely (substring), so headers like 'Image Link (...)' still map correctly. The public per-person endpoint returns only name and role - never the Aadhaar or email.

db.js

The Postgres connection pool and schema bootstrap. Requires `DATABASE_URL`. On startup it creates the visitors, sessions and people tables if they do not exist (and adds the `image_url` column to visitors if missing). SSL is on by default; set `PGSSL=disable` only for a local DB.

schema.sql

A reference copy of the same database schema that `db.js` creates automatically.

package.json

Node dependencies: express, cors, pg. Node 18+.

.env.example

Template for local configuration: DATABASE_URL, PORT, PGSSL, and ADMIN_KEY.

6.3 Web pages (registration/)

index.html

The phone self-registration page. If opened with a ?session= token it registers and confirms; otherwise it issues a standalone printable QR pass.

admin.html

The admin roster page (opened at /admin). Parses an uploaded .xlsx entirely in the browser (SheetJS), previews the rows (Aadhaar masked), then asks the server to create a person per row and returns a personal login link for each. Requires the admin key.

scan.html

The per-person login page served at /p/<token>. Greets the person by name, and on 'Proceed to login' opens the phone camera (html5-qrcode), scans the kiosk's QR, extracts the session token, and claims the session for that person.

6.4 Configuration & documentation

render.yaml

The hosting blueprint: one Node web service (root directory server/) plus a Postgres database, wired together automatically.

PROJECT.md / SETUP.md / FAQ.md

Project documentation: overview and decisions (PROJECT), all install/run/deploy/admin/pipeline commands (SETUP), and a question-and-answer guide (FAQ).

7. Database Schema

Three tables in Postgres (created automatically on server start):

visitors - a person materialised at claim time

id	TEXT PRIMARY KEY	short code
name	TEXT	
email	TEXT	
age	INTEGER	
image_url	TEXT	photo URL copied from the roster
created_at	TIMESTAMPTZ	

sessions - a kiosk-initiated session

token	TEXT PRIMARY KEY	shown in the kiosk QR
status	TEXT	'pending' 'claimed'
visitor_id	TEXT	set when claimed
created_at	TIMESTAMPTZ	
claimed_at	TIMESTAMPTZ	

people - the admin-loaded roster

token	TEXT PRIMARY KEY	the per-person login link token
name	TEXT	
role	TEXT	
aadhar	TEXT	sensitive; never exposed publicly
email	TEXT	
image_url	TEXT	direct image URL
created_at	TIMESTAMPTZ	

8. API Endpoints (server.js)

GET	/api/health	liveness check -> {ok:true}
POST	/api/sessions	kiosk starts a session -> {token}
GET	/api/sessions/:token	kiosk polls -> pending claimed + visitor
POST	/api/visitors	self-register {name,email,age,session?}
GET	/api/visitors/:id	legacy standalone-pass lookup
POST	/api/admin/upload	(needs x-admin-key) create people, return links
GET	/api/people/:token	returns name + role only
POST	/api/claim	{person, session} -> link + claim the session
GET	/admin	the admin roster page
GET	/p/:token	the per-person login page

9. Environment Variables

Kiosk app

DIYA_API_BASE	backend URL the kiosk talks to
DIYA_PIPELINE_SCRIPT	path to run1.sh (default: bundled scripts/run1.sh)
DIYA_BASH	shell used to run the pipeline (default: bash)
DIYA_REPORT_DIR	where to find the PDF (default: /opt/meditation-app/data)

Backend server

DATABASE_URL	Postgres connection string (required)
PORT	server port (default 3000)
PGSSL	set to 'disable' only for a local non-SSL Postgres
ADMIN_KEY	protects /admin uploads (without it, admin returns 503)

10. Deployment

Backend (hosting)

- Deployed from render.yaml: a Node web service (root server/) + a Postgres database.
- ADMIN_KEY must be set on the service, or the admin roster upload is disabled.
- The service must track the main branch; the phone camera scanner requires HTTPS (the host provides it).
- Free hosting sleeps when idle (first request is slow) and the free database is temporary - a persistent Postgres is recommended for anything lasting.

Kiosk (.deb)

- Build with deploy/build-deb.sh for amd64 or arm64; install with the system package tool.
- IMPORTANT: editing the C# source does not update a prebuilt .deb - the package must be rebuilt and reinstalled.
- IMPORTANT: reinstalling only takes effect after the running app is stopped first (the files are replaced on disk but the old process keeps running until restarted).
- The target machine must have: python3.10, the ~/Desktop/mark1 camera scripts, the meditation-app package installed, and /opt/meditation-app/data writable by the kiosk user.

11. Tasks Left / Open Items

The current blocker - window management

The external meditation-app has its own on-screen window and must be in front for the whole session; a stray terminal window also currently appears. Our app does not yet yield the screen to it. The planned fix: when the kiosk starts run1.sh it should step back (minimise / drop behind) so meditation-app owns the screen, then return to the front to display the report when run1.sh finishes - and the stray terminal should be suppressed.

Other open items

- Confirm meditation-app behaviour: that it blocks until the session is fully done, exits cleanly, and has finished writing the PDF before it exits (our runner shows the report only after run1.sh exits).
- run1.sh has infinite retry loops and no timeout - a permanently failing camera/script would hang the kiosk with no error screen. Consider an overall timeout and a failure screen.
- The report is displayed but is not personalised to the visitor - meditation-app is not told who logged in.
- No consent / privacy screen, even though cameras are used and Aadhaar numbers are stored.
- No inactivity auto-reset if a visitor walks away in the middle of a session.
- On-screen PDF rendering and the arm64 build have not been verified on the real kiosk hardware yet.
- Prebuilt packages are kept in the repository; producing them through an automated build and distributing them as releases would be cleaner.
- The version string is centralised in the project file but still set by hand in the build script and documentation.